Algorithms 2

Complexity

Daniel Rosenberg & Michael Trushkin

Ariel University

0.1 Polynomial-time algorithms

- An algorithm is called *polynomial-time* if its running time is bounded by $O(n^c)$ where n is the length of the input and c is some (maybe huge) constant.

0.1 Polynomial-time algorithms

- An algorithm is called *polynomial-time* if its running time is bounded by $O(n^c)$ where n is the length of the input and c is some (maybe huge) constant.

Example. Common examples of polynomial-time algorithms include: DFS, BFS, Dijkstra's algorithm, 2-Coloring, and various sorting algorithms.

0.1 Polynomial-time algorithms

- An algorithm is called *polynomial-time* if its running time is bounded by $O(n^c)$ where n is the length of the input and c is some (maybe huge) constant.

Example. Common examples of polynomial-time algorithms include: DFS, BFS, Dijkstra's algorithm, 2-Coloring, and various sorting algorithms.

Definition 1.1 $\mathbf{P}$:= The set of problems that have a polynomial algorithm.

0.2 Self reduction

- There are two types of probles:

0.2 Self reduction

- There are two types of probles:
  - *decision problems*

0.2 Self reduction

- There are two types of probles:
  - *decision problems*
  - *search problems*

0.2 Self reduction

- There are two types of probles:
  - *decision problems*
  - *search problems*
- Decision problems are those that require a ‘yes’ or ‘no’ answer

0.2 Self reduction

- There are two types of probles:
  - *decision problems*
  - *search problems*
- Decision problems are those that require a ‘yes’ or ‘no’ answer
- Search problems require finding an actual solution if one exists.

0.2 Self reduction

- There are two types of probles:
  - *decision problems*
  - *search problems*
- Decision problems are those that require a 'yes' or 'no' answer
- Search problems require finding an actual solution if one exists.

Example. If the problem is finding a path between two given nodes A and B. The decision problem will be "**Is** there a path between node A and node B?" and the search problem will be "**What** is the path between node A and B?" both of these can be solved by the same algorithm.

0.3 Self reduction

Decision:

Question 3.1 K-CLIQUE: Is there a clique of size k in G?

Search:

Question 3.2 SEARCH-k-CLQIUE: What is the clique of size k in G?

0.3 Self reduction

Decision:

Question 3.1 K-CLIQUE: Is there a clique of size k in G?

Search:

Question 3.2 SEARCH-k-CLQIUE: What is the clique of size k in G?

Claim 3.3 If the decision problem for k-clique can be solved in polynomial time, then there is a polynomial-time algorithm for SEARCH-k-CLIQUE.

0.4 proof of claim Claim 3.3

Algorithm 1:

```
1:  procedure SELF-REDUCTION(G)
2:      if A(G) = 0 then
3:          return null
4:      end
5:
6:      while v(G) > k do
7:          pick v ∈ V(G)
8:          if A(G − v) = 1 then
9:              G ← G − v
10:         end
11:     end
12:     return G
13: end
```

0.5 NP-completeness

- While the class $\mathbf{P}$ contains a large portion of the problems students have faced so far, as it turns out the majoriy of the problems are not easy at all.

0.5 NP-completeness

- While the class $\mathbf{P}$ contains a large portion of the problems students have faced so far, as it turns out the majoriy of the problems are not easy at all.

Definition 5.1 (NP class) A language L is said to be in $\mathbf{NP}$ if we have a polynomial-time algorithm M such that

$$x \in L \Leftrightarrow \exists y \ s.t \ |y| < p(|x|) \text{ and } M(x, y) = 1$$

where p is some polynomial

- In most literture y is called a *witness* and M is called *veryfing algorithm*, where y plays the role of the answer, and M should just verify if the answer is correct.

We are ready to meet out first **NP** language:

Claim 6.1 k-CLIQUE is in **NP**

0.7 proof of claim Claim 6.1

Algorithm 2: Verifying algorithm for k-clique

```
1:  procedure M(G, Y)
2:      ▷ Check if the set of the correct size
3:      if |Y| ≠ k then
4:          return false
5:      end
6:
7:      ▷ Check that all the vertices are real
8:      for v ∈ Y do
9:          if v ∉ V(G) then
10:             return false
11:         end
12:     end
13:
14:     ▷ Check that all the edges exist
15:     for v, u ∈ Y do
16:         if (v, u) ∉ E(G) then
17:             return false
18:         end
19:     end
20:
21:     return true
22: end
```

0.7 proof of claim Claim 6.1

Algorithm 3: Verifying algorithm for k-clique

```
1:  procedure M(G, Y)
2:      ▷ Check if the set of the correct size
3:      if |Y| ≠ k then
4:          return false
5:      end
6:
7:      ▷ Check that all the vertices are real
8:      for v ∈ Y do
9:          if v ∉ V(G) then
10:             return false
11:         end
12:     end
13:
14:     ▷ Check that all the edges exist
15:     for v, u ∈ Y do
16:         if (v, u) ∉ E(G) then
17:             return false
18:         end
19:     end
20:
21:     return true
22: end
```

- if $G \in k$-clique, then there is a subset $V' \subseteq V(G)$ such that $G[V'] \cong K_k$, and $M(G, V') = 1$

0.7 proof of claim Claim 6.1

Algorithm 4: Verifying algorithm for k-clique

```
1:  procedure M(G, Y)
2:      ▷ Check if the set of the correct size
3:      if |Y| ≠ k then
4:          return false
5:      end
6:
7:      ▷ Check that all the vertices are real
8:      for v ∈ Y do
9:          if v ∉ V(G) then
10:             return false
11:         end
12:     end
13:
14:     ▷ Check that all the edges exist
15:     for v, u ∈ Y do
16:         if (v, u) ∉ E(G) then
17:             return false
18:         end
19:     end
20:
21:     return true
22: end
```

- if $G \in k$-clique, then there is a subset $V' \subseteq V(G)$ such that $G[V'] \cong K_k$, and $M(G, V') = 1$
- if $G \notin k$-clique, then no matter which subset $V' \subseteq V(G)$ we take, $G[V']$ will never be a clique in G. That is Y' either will have too many \ little vertices, have "fake" vertices or there will be some missing edges, so that $M(G, V') = 0$.

0.7 proof of claim Claim 6.1

Algorithm 5: Verifying algorithm for k-clique

```
1:  procedure M(G, Y)
2:      ▷ Check if the set of the correct size
3:      if |Y| ≠ k then
4:          return false
5:      end
6:
7:      ▷ Check that all the vertices are real
8:      for v ∈ Y do
9:          if v ∉ V(G) then
10:             return false
11:         end
12:     end
13:
14:     ▷ Check that all the edges exist
15:     for v, u ∈ Y do
16:         if (v, u) ∉ E(G) then
17:             return false
18:         end
19:     end
20:
21:     return true
22: end
```

- if $G \in k$-clique, then there is a subset $V' \subseteq V(G)$ such that $G[V'] \cong K_k$, and $M(G, V') = 1$
- if $G \notin k$-clique, then no matter which subset $V' \subseteq V(G)$ we take, $G[V']$ will never be a clique in G. That is Y' either will have too many \ little vertices, have "fake" vertices or there will be some missing edges, so that $M(G, V') = 0$.
- The algorithm runs in $O(2k) + O(k^2)$ time which is polynomial.

0.8 Reductions

- Suppose we have two languages/problems L_1, L_2, can we know which one of them is *harder*?

0.8 Reductions

- Suppose we have two languages/problems L_1, L_2, can we know which one of them is *harder*?
- The intuition is that if L_2 harder than L_1 we would be able to solve L_2 using L_1.

0.8 Reductions

- Suppose we have two languages/problems L_1, L_2, can we know which one of them is *harder*?
- The intuition is that if L_2 harder than L_1 we would be able to solve L_2 using L_1.
- This is done by "translating" our problem from L_1 to L_2, solving the translated L_1 problem, and then answering accordingly.

0.8 Reductions

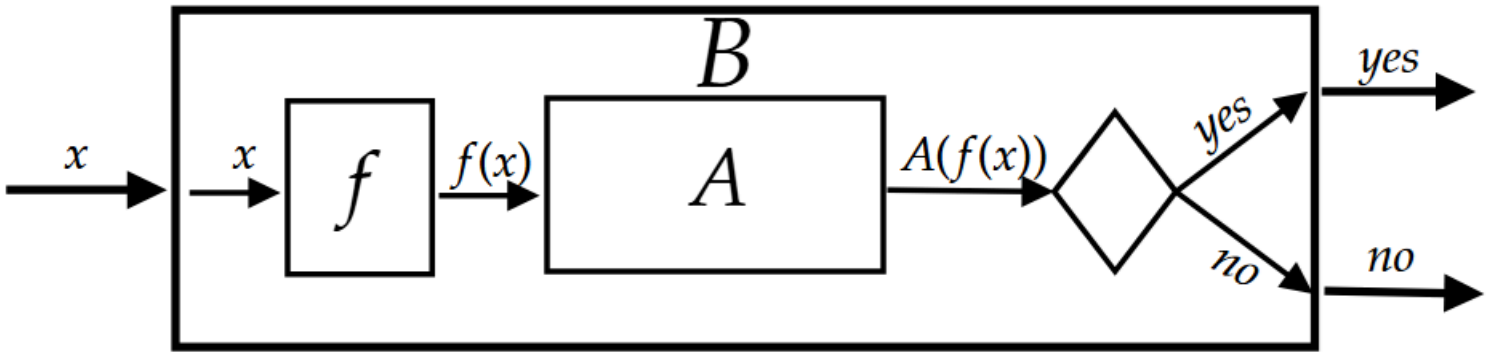
- Suppose we have two languages/problems L_1, L_2, can we know which one of them is *harder*?
- The intuition is that if L_2 harder than L_1 we would be able to solve L_2 using L_1.
- This is done by "translating" our problem from L_1 to L_2, solving the translated L_1 problem, and then answering accordingly.

Definition 8.1 (polynomial time reduction) Given two languages $L_1, L_2 \in \mathbf{NP}$, we write $L_1 \leq_p L_2$ if there exists a function $f : \{0,1\}^* \to \{0,1\}^*$ and a polynomial $p : \mathbb{N} \to \mathbb{N}$, such that:

- $x \in L_1 \Leftrightarrow f(x) \in L_2$
- for every $x \in \{0,1\}^*$, f runs in $p(|x|)$ time.

0.9 Reductions

Assuming that $L_1 \leq_p L_2$ and given the polynomial reduction f as well as a black box A that solves L_2. We can create an algorithm B that solves L_2 using only f and A in the following way:



0.10 Reductions

Definition 10.1 (NP-hard) A language $L \subseteq \{0,1\}^*$ is said to be NP-hard if $L' \leq_p L$ for every $L' \in \mathbf{NP}$

0.10 Reductions

Definition 10.1 (NP-hard) A language $L \subseteq \{0,1\}^*$ is said to be NP-hard if $L' \leq_p L$ for every $L' \in \mathbf{NP}$

Intuitively, the following qustion arises:

Question 10.2 Are there any languages that are NP-Hard?

0.10 Reductions

Definition 10.1 (NP-hard) A language $L \subseteq \{0,1\}^*$ is said to be NP-hard if $L' \leq_p L$ for every $L' \in \mathbf{NP}$

Intuitively, the following qustion arises:

Question 10.2 Are there any languages that are NP-Hard?

Example. The language

$$L^* = \left\{(M', x, 1^c) : M'(x') = 1 \wedge M'(x') \text{ computes in } O\left(2^{|x|^c}\right) \text{ time}\right\}$$

is NP-hard.

0.11 Reductions

Definition 11.1 (NP-complete) A language $L \subseteq \{0,1\}^*$ is said to be NP-complete if $L \in \mathbf{NP}$ and L is NP-hard

- If we find a polynomial time algorithm for one problem in $\mathbf{NPC}$, then $\mathbf{P} = \mathbf{NP}$
- The problem $\mathbf{P} = \mathbf{NP}$ or $\mathbf{P} \neq \mathbf{NP}$ remains an open question to this day. Whoever proves either one will be awarded 1 million dollars.

0.11 Reductions

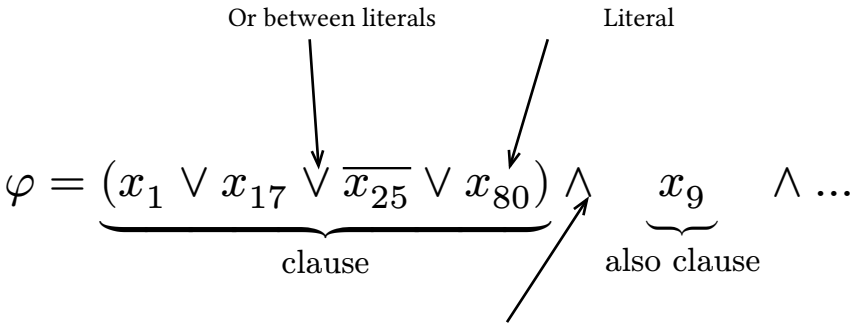
Definition 11.1 (NP-complete) A language $L \subseteq \{0,1\}^*$ is said to be NP-complete if $L \in \mathbf{NP}$ and L is NP-hard

- If we find a polynomial time algorithm for one problem in $\mathbf{NPC}$, then $\mathbf{P} = \mathbf{NP}$
- The problem $\mathbf{P} = \mathbf{NP}$ or $\mathbf{P} \neq \mathbf{NP}$ remains an open question to this day. Whoever proves either one will be awarded 1 million dollars.

Question 11.2 Is there any language $L \in \mathbf{NPC}$?

0.12 SAT

- Let $x_1, \ldots x_n$ be boolean variables (x_i can be assigned either 0 or 1).
- A boolean formula φ is said to be in conjunctive normal form (CNF) if it has the form



Theorem 12.1 (Cook-Levin) CNF-SAT is npc.

0.13 3-CNF

Definition 13.1 For an integer $k \in \mathbb{N}$,

$$k\text{-CNF-SAT} := \{\varphi \mid \varphi \text{ is a CNF formula in which each clause has exactly } k \text{ literals}\}.$$

0.13 3-CNF

Definition 13.1 For an integer $k \in \mathbb{N}$,

$$k\text{-CNF-SAT} := \{\varphi \mid \varphi \text{ is a CNF formula in which each clause has exactly } k \text{ literals}\}.$$

Theorem 13.2 2-CNF-SAT is in $\mathbf{P}$.

- proof is delegated to the practice session

Theorem 13.3 3-CNF-SAT is in $\mathbf{NPC}$.

0.14

Claim 14.1 3-CNF-SAT$\in$ **NP**

Claim 14.1 3-CNF-SAT$\in$ **NP**

Algorithm 7: Verifying algorithm for 3-CNF-SAT

```
1:  procedure M(φ, α)
2:      ▷ α is the assigment i.e α : {x_1, ..., x_n} → {T, F}
3:      ▷ Check if the assignment complete, i.e. every variable has an assigment
4:      for i ∈ [n] do
5:          if α(x_i) = undefined then
6:          end
7:      end
8:  end
9:  ▷ Check if any clause is unsatisfied
10: for clause C ∈ φ do
11:     if C(α) = F then
12:         return false
13:     end
14: end
15: ▷ All clases are satisfied, hence φ(α) is also satisfied
16: return true
```

0.15

- Key observasion:
  - If $L_1 \leq_p L_2$ and $L_2 \leq_p L_3$, then $L_1 \leq_p L_3$.

Claim 15.1 Let $L \in$ NP-Hard, and let L' be a language. If $L \leq_p L'$, then L' is also NP-Hard.

Claim 16.1 3-CNF-SAT is NP-hard

- Define a function f as follows:

$$f(\varphi) = \bigwedge_{i=1}^{m} g(C_i)$$

1. If $k = 3$, then return C as it. i.e.

$$g(l_1 \vee l_2 \vee l_3) = l_1 \vee l_2 \vee l_3$$

2. If $k < 3$, then repeat one of the literals until the clause has exactly 3 literals. For example

$$g(l_1 \vee l_2) = l_1 \vee l_2 \vee l_2.$$

3. If $k > 3$ then create $k - 3$ **new** variables named $y_1, \ldots, y_{k-3}$ and let:

$$g(l_1 \vee l_2 \vee \ldots l_k) = (l_1 \vee l_2 \vee y_1) \wedge (\overline{y_1} \vee l_3 \vee y_2) \wedge (\overline{y_2} \vee l_4 \vee y_3) \wedge \ldots \wedge (\overline{y_{k-3}} \vee l_{k-1} \vee l_k).$$

0.17

- It is clear that f runs in constant time.

0.17

- It is clear that f runs in constant time.
- It remains to show the correctness of f, i.e.

$$\varphi \in \text{CNF-SAT} \Leftrightarrow f(\varphi) \in 3 - \text{CNF-SAT}$$

0.17

- It is clear that f runs in constant time.
- It remains to show the correctness of f, i.e.

$$\varphi \in \text{CNF-SAT} \Leftrightarrow f(\varphi) \in 3 - \text{CNF-SAT}$$

$\Rightarrow$(Completeness):

- Assume that $\varphi \in$ CNF-SAT,
- This means there exists a satisfying assignment α for φ.
- set α' as follow:

$$\forall i \in [n]: \quad \alpha'(x_i) = \alpha(x_i).$$

- Fix a clause $C \in \varphi$
- If C we have $|C| \leq 3$, then we are done as $C \equiv g(C)$.

0.18

- Otherwise let $C = l_1 \vee l_2 \vee \ldots \vee l_k$ where $k \geq 4$ be such clause.
- By assumption $C[\alpha] = T$, so there exists some $i \in [k]$ such that $\ell_i = T$.
- Since for all $j \in [k-3]$ the value of $\alpha'(y_j)$ have not yet been set, we define them as follows:

$$\alpha'(y_j) = \begin{cases} T & \text{if } \ j \leq i-2, \\ F & \text{if } \ j > i-2. \end{cases}$$

Then,

$$\begin{aligned} g(C)[\alpha'] &= (l_1 \vee l_2 \vee y_1) \wedge \ldots \wedge (\overline{y_{i-2}} \vee l_i \vee y_{i-1}) \wedge \ldots \wedge (\overline{y_{k-3}} \vee l_{k-1} \vee l_k)[\alpha'] \\ &= (? \ \vee ? \ \vee T) \wedge \ldots \wedge (F \vee T \vee T) \wedge \ldots \wedge (T \vee ? \vee ?) = T. \end{aligned}$$

as required.

0.19

$\Leftarrow$(Soundness):

0.19

$\Leftarrow$(Soundness):

- Assume that $f(\varphi) \in$ 3-CNF-SAT.
- there is a satisfying assignment α' for $f(\varphi)$.

0.19

$\Leftarrow$(Soundness):

- Assume that $f(\varphi) \in$ 3-CNF-SAT.
- there is a satisfying assignment α' for $f(\varphi)$.
- copy the assignment of the orginal variables form α' to α.

0.19

$\Leftarrow$(Soundness):

- Assume that $f(\varphi) \in$ 3-CNF-SAT.
- there is a satisfying assignment α' for $f(\varphi)$.
- copy the assignment of the orginal variables form α' to α.
- if α is not sattisfying then, there must exists a clause $C = l_1 \vee l_2 ... \vee l_k$.
- it cannot be the $k \leq 3$ as we are coppying the assigment.
- for $k > 3$ by assumption

$$C[\alpha] = l_1 \vee l_2 \vee ... \vee l_k = F$$

such that $C[\alpha] = F$.

0.19

$\Leftarrow$(Soundness):

- Assume that $f(\varphi) \in$ 3-CNF-SAT.
- there is a satisfying assignment α' for $f(\varphi)$.
- copy the assignment of the orginal variables form α' to α.
- if α is not sattisfying then, there must exists a clause $C = l_1 \vee l_2 ... \vee l_k$.
- it cannot be the $k \leq 3$ as we are coppying the assigment.
- for $k > 3$ by assumption

$$C[\alpha] = l_1 \vee l_2 \vee ... \vee l_k = F$$

such that $C[\alpha] = F$.

- If $k \leq 3$, then since $g(C) \equiv C$, it follows that $T = g(C)[\alpha'] = C[\alpha'] = C[\alpha] = F$ which is a contradiction to the assumption that α satisfies $f(\varphi)$.

0.19

$\Leftarrow$(Soundness):

- Assume that $f(\varphi) \in$ 3-CNF-SAT.
- there is a satisfying assignment α' for $f(\varphi)$.
- copy the assignment of the orginal variables form α' to α.
- if α is not sattisfying then, there must exists a clause $C = l_1 \vee l_2 ... \vee l_k$.
- it cannot be the $k \leq 3$ as we are coppying the assigment.
- for $k > 3$ by assumption

$$C[\alpha] = l_1 \vee l_2 \vee ... \vee l_k = F$$

such that $C[\alpha] = F$.

- If $k \leq 3$, then since $g(C) \equiv C$, it follows that $T = g(C)[\alpha'] = C[\alpha'] = C[\alpha] = F$ which is a contradiction to the assumption that α satisfies $f(\varphi)$.
- Otherwise assume that $k \geq 4$, by assumption

$$C[\alpha] = l_1 \vee l_2 \vee ... \vee l_k = F$$

0.19

$\Leftarrow$(Soundness):

- Assume that $f(\varphi) \in$ 3-CNF-SAT.
- there is a satisfying assignment α' for $f(\varphi)$.
- copy the assignment of the orginal variables form α' to α.
- if α is not sattisfying then, there must exists a clause $C = l_1 \vee l_2 ... \vee l_k$.
- it cannot be the $k \leq 3$ as we are coppying the assigment.
- for $k > 3$ by assumption

$$C[\alpha] = l_1 \vee l_2 \vee ... \vee l_k = F$$

such that $C[\alpha] = F$.

- If $k \leq 3$, then since $g(C) \equiv C$, it follows that $T = g(C)[\alpha'] = C[\alpha'] = C[\alpha] = F$ which is a contradiction to the assumption that α satisfies $f(\varphi)$.
- Otherwise assume that $k \geq 4$, by assumption

$$C[\alpha] = l_1 \vee l_2 \vee ... \vee l_k = F$$

meaning that for all $i \in [k]$ we have $l_i = F$.

0.19

- On the otherhand, since $f(\varphi)[\alpha']$ is satisfied the gadget clause

$$g(C) = (l_1 \vee l_2 \vee y_1) \wedge (\overline{y_1} \vee l_3 \vee y_2) \wedge (\overline{y_2} \vee l_4 \vee y_3) \wedge ... \wedge (\overline{y_{k-3}} \vee l_{k-1} \vee l_k)$$

0.19

- On the otherhand, since $f(\varphi)[\alpha']$ is satisfied the gadget clause

$$g(C) = (l_1 \vee l_2 \vee y_1) \wedge (\overline{y_1} \vee l_3 \vee y_2) \wedge (\overline{y_2} \vee l_4 \vee y_3) \wedge \ldots \wedge (\overline{y_{k-3}} \vee l_{k-1} \vee l_k)$$

is also satisfied, and such every clause should be T.

- In order for the first clause to be T, $y_1 = T$ must hold.
- In order to satisfy the next clause $y_2 = T$ must hold, following this argument one can see that $y_i = T$ must hold for all $i \in [k-3]$.
- Looking at the last clause, $\ell_{k-1} = \ell_k = F$ by assumption, and $\overline{y_{k-3}} = F$ in order for any other clause to be satisfied,
- which is a contradiction to the assumption that a' satisfies φ.